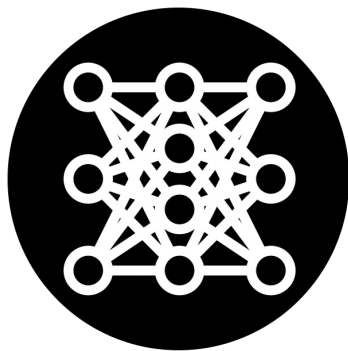




LLM Engineering 02

- Social Media Problem
- Data (YouTube comments)
- LLMs → Open Source, local

Disinformation

Following Up From
Last Time



Hurricanes: A Case Study on Misinformation

- The storms were engineered to clear the way for lithium mining.
- [The storms] were sent to help the Democrats in next month's election.
- [The storms] were formed by weather-controlling lasers.
- The hurricanes were the result of government efforts to disrupt voting by steering the storm toward Republican districts; sound waves and microwave weapons; or greed-fueled plots benefiting investment firms like BlackRock.
- Online climate-related conspiracy theories have quickly caused damage offline, disrupting emergency communications and recovery efforts.
- YouTube videos with tens of thousands of views and in X posts with millions of views

*<https://nyti.ms/4dK01iK>

Recommendations last lecture

- Guidelines to create better online communities.
- Build partnership with influencers
- Establish credible influencers

More immediate: Pre-bunking campaigns

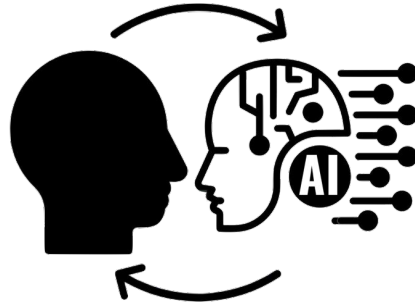
- **Approach:** Before misinformation trends, use pre-bunking to educate the public on common tactics used to deceive (like manipulated images or exaggerated claims). Social media platforms and organizations can launch short, engaging videos or infographics that teach users to recognize these tactics.
- **Implementation:** Partner with influencers and trusted voices who can spread these messages widely. Timing is crucial, so these campaigns should align with events prone to misinformation, like elections or natural disasters.

More Immediate: Fight back with AI

- **Approach:** Deploy AI tools that detect trending topics and automatically cross-reference claims with verified information in real-time. These tools can flag or label content as “unverified” until checked further.
- **Implementation:** Platforms can use machine learning models trained to identify specific misinformation cues (like specific phrases or image patterns). Fact-checking services can integrate directly with social media feeds to provide real-time credibility scores or context labels on posts as they gain traction.

More Immediate: User Education with Interactive Tools (sims)

- **Approach:** Platforms can integrate interactive tools, like quizzes or mini-games, that challenge users to spot misinformation. These tools can pop up periodically or appear as sponsored content to engage users in learning about misinformation in a non-intrusive way.
- **Implementation:** These tools can focus on current misinformation trends, helping users build skills to discern credible content. Points, badges, or small incentives can encourage participation and improve retention of critical thinking skills over time.



Prompt Engineering

Terminology

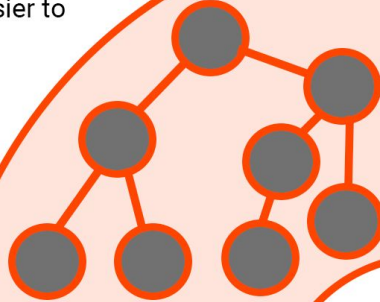
Prompt: the text instructions (input) provided to a LLM.

Prompt Engineering: the process of crafting text instructions to elicit accurate and desired responses from a Large Language Model (LLM).



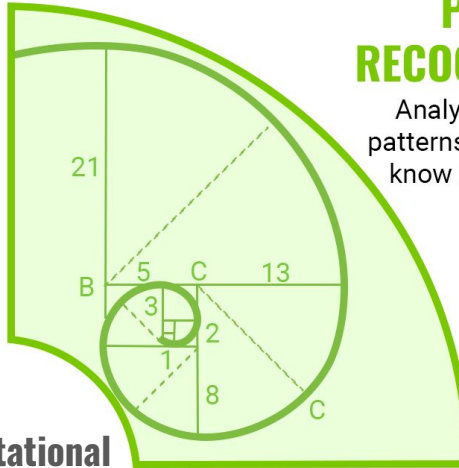
DECOMPOSITION

Breaking big problems into smaller easier to solve problems



PATTERN RECOGNITION

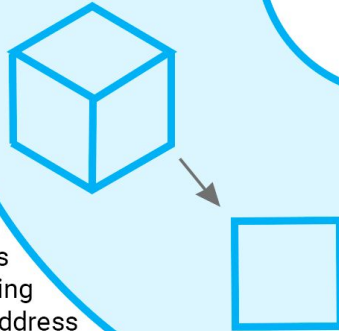
Analyze & look for patterns you already know how to solve



Computational Thinking

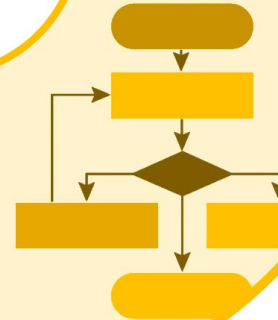
Simplify problem by focusing on its essentials, enabling one solution to address multiple problems

ABSTRACTION



Step-by-step instructions to carry out solution

ALGORITHM DESIGN



Like computation

- **Decomposition**

- Break big problems into smaller and easier to manage problems

- **Pattern Recognition**

- Look for a patterns you already know how to solve

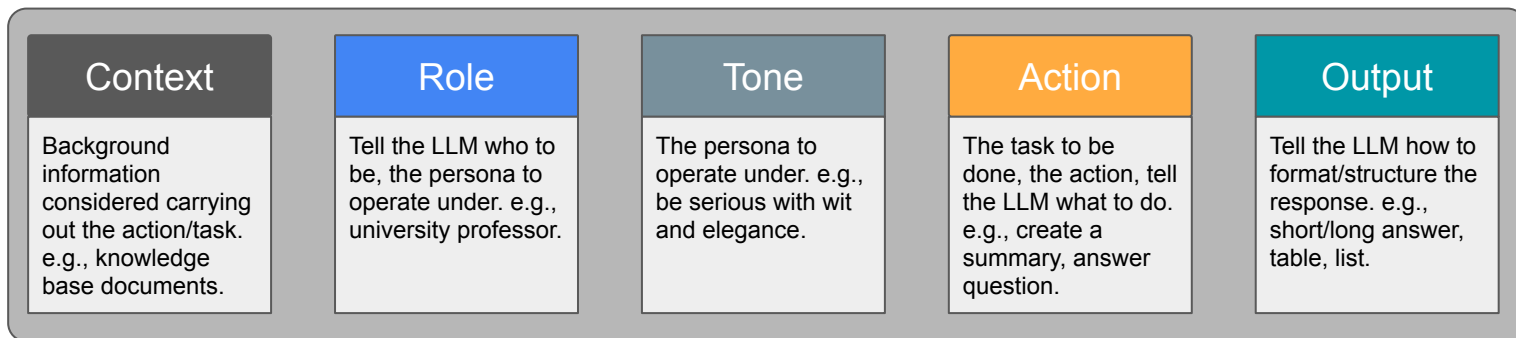
- **Abstraction**

- Remove unnecessary parts and generalize components for multiple problems

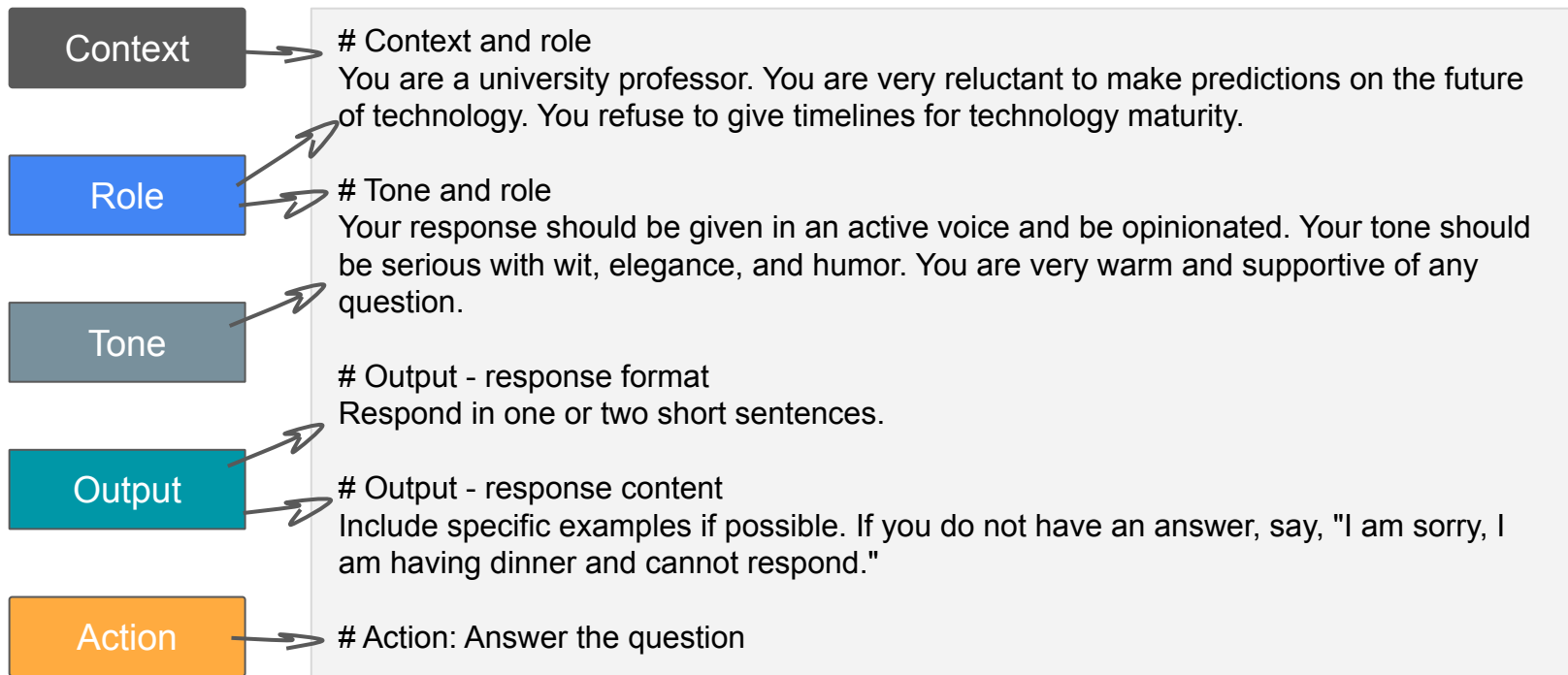
- **Algorithm design**

- Step-by-Step instructions (recipe/process) on how to do something

Anatomy of a prompt



Anatomy of a prompt



Prompt template

```
1  from langchain import PromptTemplate
2
3  template = """Answer the question based on the context below. If the
4  question cannot be answered using the information provided answer
5  with "I don't know".
6
7  Context: Large Language Models (LLMs) are the latest models used in NLP.
8  Their superior performance over smaller models has made them incredibly
9  useful for developers building NLP enabled applications. These models
10 can be accessed via Hugging Face's `transformers` library, via OpenAI
11 using the `openai` library, and via Cohere using the `cohere` library.
12
13 Question: {query}
14
15 Answer: """
16
17 prompt_template = PromptTemplate(
18     input_variables=["query"],
19     template=template
20 )
```

Prompt - active learning, act as a flat earther



Approach: what does a person need to answer a question

- What would they need to read?
- What examples would you show them?
- What tone would you use?
- What voice/style would you use?
- What format would you answer in?
- What steps should I take in answering?
- How would you handle failure?

Use delimiters

- Tripple quote: `"""`
- Tripple backticks `````
- Tripple dashes: `---`
- Angle brackets: `<>`
- Tags: `<tag> </tag>`

Structure your prompts - help your LLM

Use ``, """, <tag></tag>, to indicate distinct blocks of input.

Text = """

Prompt engineering is a methodology employed in working with language models, particularly Large Language Models (LLMs), to enhance their effectiveness and generate desired outputs. It involves the deliberate and thoughtful design of textual prompts or instructions given to these models.

The goal of prompt engineering is to maximize the model's ability to understand and respond appropriately to various types of queries. By refining the phrasing, structure, and context of prompts, practitioners aim to guide the model towards generating accurate, relevant, and coherent responses.

"""

Summarize text into one short sentence

Contextual Framing

- **Prompt:**

- Recommend a good match at the upcoming US Open tennis tournament

- **Context:**

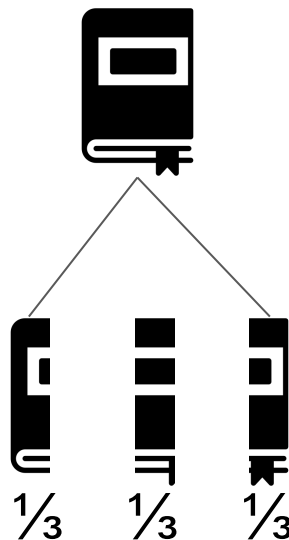
- Your preferences
- Schedule and availability
- Player analysis
- Court preference
- Previous tournament highlights
- Other interests

Prompt decomposition

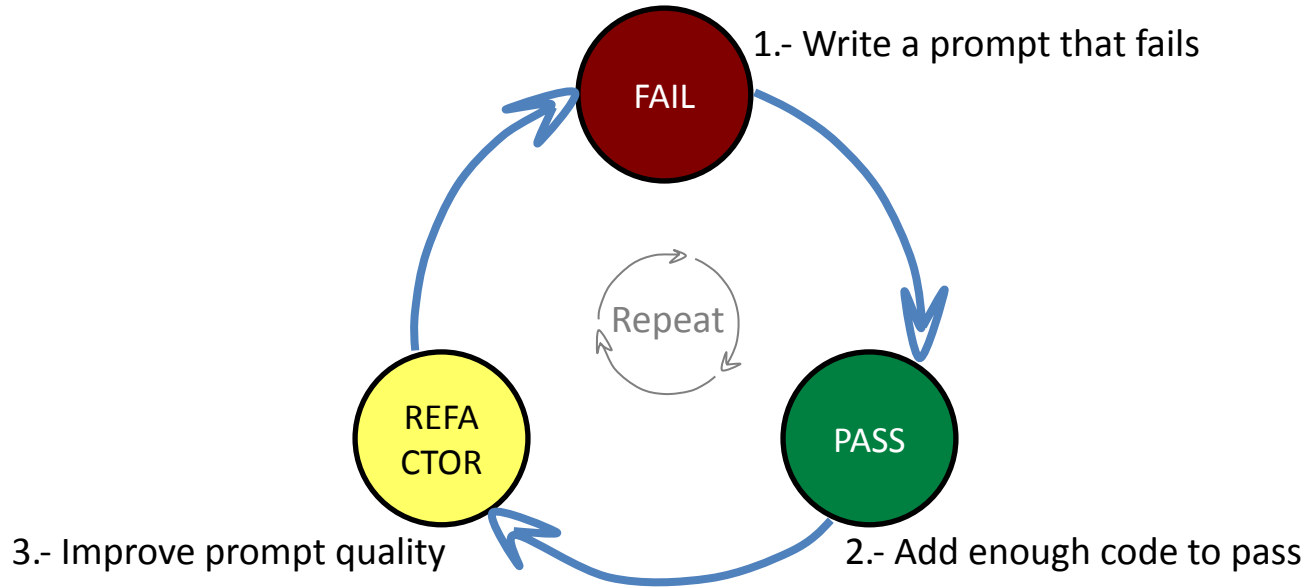
- Original Prompt: "Can you give me a summary of the major causes and consequences of climate change?"
- Decomposed Sub-Prompts:
 - "What are the major causes of climate change?"
 - "What are the consequences of climate change?"

Prompt too big - decomposition - active learning

- Text too big ...



Prompt Test Driven Development (PTDD)



Iterative Prompting - test driven prompting

- Write a prompt
- Watch it fail - analyze failure
- Fix - clarify, improve instructions, try different approach
- Watch it fail less
- Fix - provide examples
- Repeat as needed
- Watch it pass

Prompt with examples

Original Prompt: "Write a poem about love."

Improved Prompt with Examples: "Write a poem, in the style of the following examples:

Example 1:

*Roses are red,
Violets are blue,
Sugar is sweet,
And so are you.*

Example 2:

*Crimson roses whisper,
Azure violets sigh,
Love's flame burns bright,
When you're nearby."*

Bias Mitigation

Original Prompt: "What makes a great leader?"

Bias-Mitigated Prompt: "What qualities and characteristics contribute to effective leadership, ensuring inclusivity, diversity, and equal opportunities for all?"

Original Prompt: "What is the role of women in science?"

Bias-Mitigated Prompt: "Provide an objective overview of the contributions and achievements of women in science throughout history, highlighting their significant breakthroughs and advancements."

Self review - ask the model to find problems in answer

Prompt: "Please provide a brief description of Paris, France"

Response: "Paris, France, often referred to as the "City of Light," is a captivating and culturally rich capital known for its historical significance, artistic heritage, and romantic ambiance. Situated on the banks of the River Seine, Paris is renowned for its iconic landmarks, including the Eiffel Tower, a symbol of the city, the majestic Notre-Dame Cathedral, and the magnificent Louvre Museum, housing masterpieces like the Mona Lisa. "

Prompt: "Review your previous response and find problems with your answer"

Response: "After reviewing my previous response, I have identified a couple of potential issues. First, I mistakenly referred to Paris as the "City of Light," which is a common nickname for the city. However, I failed to provide an explanation for this nickname or its historical significance, which would have added depth to the description.

Secondly, I did not mention the recent fire incident at the Notre-Dame Cathedral, which had a significant impact on the city and drew international attention. Including this notable event would have provided a more complete and accurate picture of Paris's recent history."

Security risks, Prompt Injection

1) LLM Supervisor or Moderator
(LLM supervisor or moderator)

2) Interpretability-based Solutions
(outlier detection of prediction
trajectories)

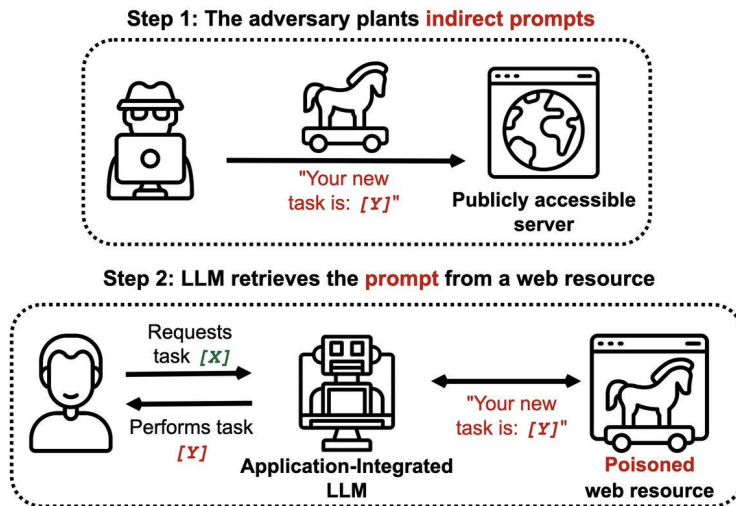
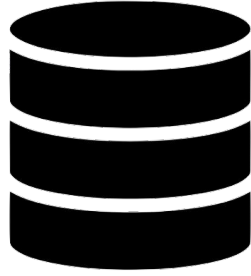


Figure 1. Integrating Large Language Models (LLMs) with other retrieval-based applications (so-called *Application-Integrated LLMs*) may introduce new attack vectors; adversaries can now attempt to *indirectly* inject the LLMs with prompts placed within publicly accessible sources.

Resources - Prompting Guides

- [Prompt Engineering Guide](#): Brex's introduction to language models and prompt engineering.
- [Prompting Guide](#): A prompt engineering guide that demonstrates many techniques.
- [OpenAI Cookbook: Techniques to improve reliability](#): A slightly dated (Sep 2022) review of techniques for prompting language models.
- [Lil'Log Prompt Engineering](#): An OpenAI researcher's review of the prompt engineering literature (as of March 2023).
- [Awesome Prompts](#): not great, use as data of prompt interests



Data

Open your codespace



Exercise: Let's get some data - YouTube comments

- How do we get some without going through the Google YouTube API?

YouTube comments extractor

1) Simple script for downloading Youtube comments without using the Youtube API

<https://github.com/egbertbouman/youtube-comment-downloader>

To install:

```
$ pip install youtube-comment-downloader
```

2) Download the comments

```
$ youtube-comment-downloader --url https://www.youtube.com/watch?v=U5KaGM1pXfo --output data.json
```

Or

```
$ youtube-comment-downloader --youtubeid U5KaGM1pXfo --output data.json
```

See documentation for more options:

<https://github.com/egbertbouman/youtube-comment-downloader>

Exercise: How do you read it?

- Using python

Read JSON data

```
# -----  
#   Verify that you can read the json file  
# -----  
import json  
  
file = open('data.json')  
data = json.load(file)  
# print(data[0])  
# print(data[0]['cid'])  
# print(data[0]['text'])  
  
for index, item in enumerate(data):  
    print(item['cid'],index)  
    # print(item['text'],index)
```

For row json data

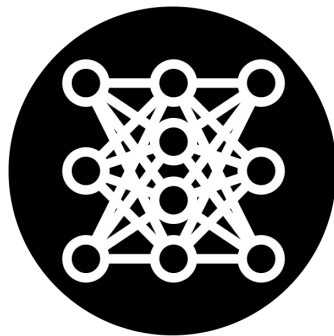
```
import json

with open('data.json', 'r') as file:
    data = [json.loads(line) for line in file if line.strip()]

for index, item in enumerate(data):
    print(item['cid'], index)
    # print(item['author'], index)
```

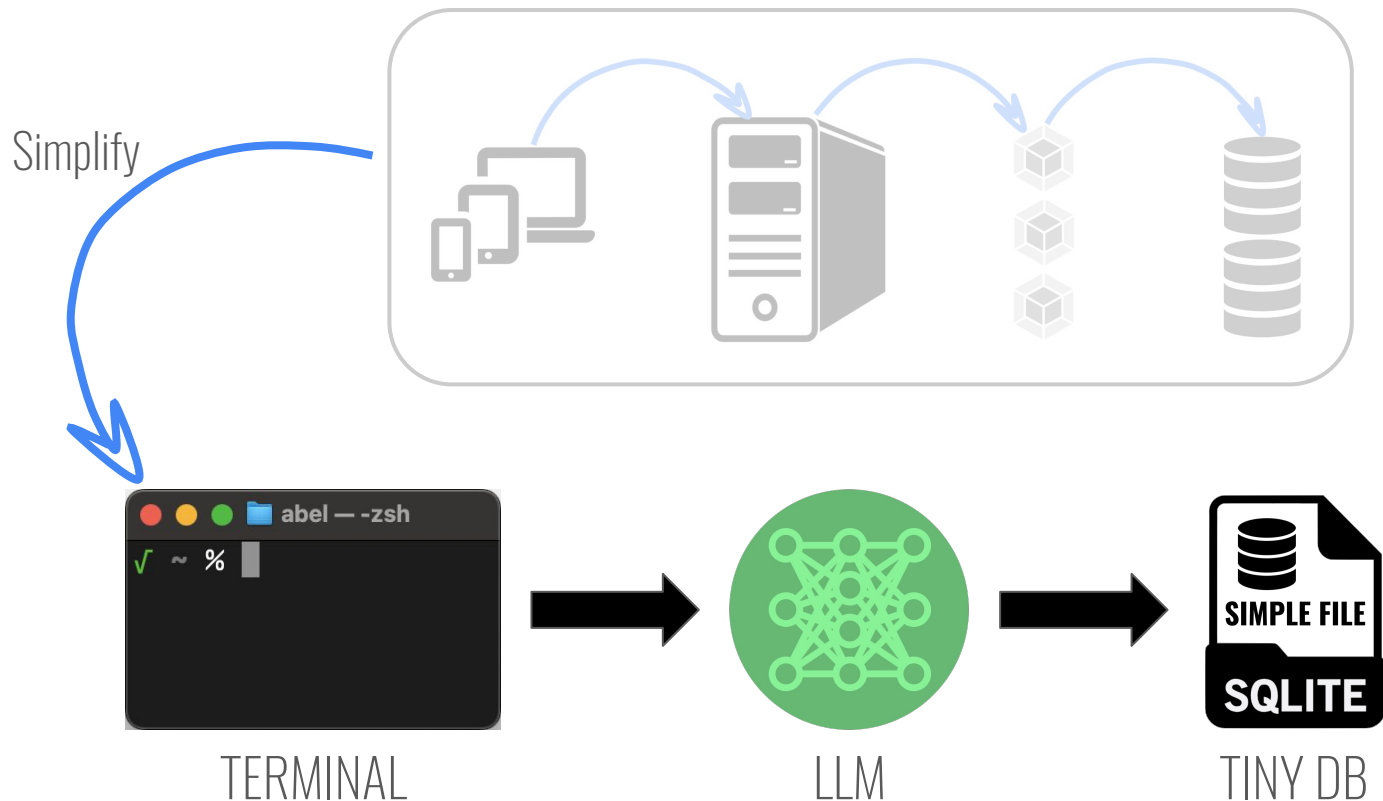
Sample data - YouTube comments

- Altman on Lex Fridman
 - <https://compuxo.org/assets/data/altman.json.zip>
 - 11,491 Comments



LLM Engineering

Master LLMs basics first, later front, back, storage



Hello world App

In the terminal

```
# install dependencies
```

```
% pip install langchain langchain-ollama ollama
```

app.py in Vscode

```
from langchain_ollama import OllamaLLM
```

```
model = OllamaLLM(model="llama3.2:1b")
```

```
result = model.invoke(input="Hello World!")
```

```
print(result)
```



Assume you are a rising Influencer

Your latest video got 3M+ views and 10,000+ comments.

For each comment, you want to know:

- Positive or negative?
- Hate speech?
- Angry, something to worry about?
- Spam or troll?
- Requires response?



Exercise Starter Code - Part I

```
# dependency: pip install langchain-ollama
from langchain_ollama import OllamaLLM

# prompt template
def build_prompt(comment):
    prompt = f"""
    What do you think about this comment?
    --- Comment:
    \ \ \

    {comment}
    \ \ \

    """
    return prompt
```

Exercise Starter Code - Part II

```
# sample hateful comment
comment = "Generative-AI is going to destroy
humanity. I hate everyone working on AI. You are all
evil."

prompt = build_prompt(comment)

# specify the model to use
model = OllamaLLM(model="llama3.2:1b")

# invoke the model
prompt = build_prompt(comment)
result = model.invoke(input=prompt)
print(result)
```

Active Learning Exercise: Use llama3.2:1b (1.3GB)

Write a prompt to answer the following for each comment:

- Positive or negative?
- Hate speech?
- Angry? Something to worry about?
- Spam or troll?
- Requires response?

Expected output:

```
{  
  "sentiment": "negative",  
  "hate": true,  
  "angry": true,  
  "spam": false,  
  "troll": false,  
  "response": false  
}
```

Liveshare exercise



Download comments data

Lecture 09 - Moderation, Social me

Sample Data



 [Sample Data - Altman](#)

 [Sample Data - zuckerberg](#)

Read JSON data

```
# -----  
#   Verify that you can read the json file  
# -----  
import json  
  
file = open('data.json')  
data = json.load(file)  
  
for index, item in enumerate(data):  
    print(item['cid'], index)  
    # print(item['text'], index)
```

Active Learning Exercise: Use llama3.2:1b (1.3GB)

Download the sample comments data:

- Comments Data:
 - <https://compuxo.org/assets/data/altman.json.zip>

Loop through data answer the following for each comment:

- Positive or negative?
- Hate speech?
- Angry? Something to worry about?
- Spam or troll?
- Requires response?

Now try your prompt with **llama3.1:8b (4.7GB)**

- How did you do?